LanceDB Architecture and Vector Indexing (Part 1)

LanceDB is an open-source, serverless vector database built on top of the Lance columnar data format.
Unlike traditional vector databases that require always-on server clusters and keep entire indexes in RAM,
LanceDB is designed to query disk-based vector indices with minimal memory overhead.

The primary indexing mechanism in LanceDB is IVF-PQ (Inverted File with Product Quantization).
IVF partitions the vector space into voronoi cells using k-means clustering.
During query execution, only the centroids closest to the query vector are visited (controlled by the nprobes paramet
Product Quantization (PQ) compresses high-dimensional floating point vectors into compact byte codes.
By dividing an original D-dimensional vector into M sub-vectors and quantizing each sub-vector into centroids,
PQ reduces vector memory footprint by a factor of 16x to 32x while maintaining high cosine similarity fidelity.

A critical feature of LanceDB is zero-copy reads via Apache Arrow. Data transfers between disk and RAM avoid
serialization penalties, enabling sub-millisecond retrieval on local SSDs and cost-effective querying against Amazon

LanceDB Disk-Based Search and Filtering (Part 2)

LanceDB supports both vector search and zero-cost scalar metadata filtering.
Scalar filters can be executed as pre-filtering (filtering candidate rows before vector distance computation) or post-filtering (filtering after top-k vector retrieval).

Pre-filtering prevents low recall caused by restrictive metadata constraints, which frequently degrades post-filtering.
Lance's columnar layout allows selective column projection: when querying for similarities, the engine only reads the vector column and the filtered metadata columns from storage, skipping unused columns entirely.

For index persistence, LanceDB stores immutable versioned manifests, supporting ACID transactions and snapshot
When documents are updated or deleted, LanceDB uses tombstone markers and merge-on-read compaction, ensuring that writes never block concurrent read queries.